队伍编号	MCB2300890
赛道	A

基于多模型调参优化的坑洼道路分类识别方法研究

摘 要

道路基础设施是至关重要的公共资产，交通发达有助于经济发展和增长，同时带来重要的社会效益。研究表明，事故率随着路面的粗糙度增加而增加，因此如何更好地维护城市道路路面的长期稳定运行十分重要，因此道路坑洼检测分类具有很强的实际应用价值和重要的意义。本文旨在建立一个**以 VisionTransformer 为主干网络，基于多模型调参优化的融合模型，对坑洼路面进行识别和分类**。

针对问题一：主要对数据集统一预处理，获取数据特征并建立模型分析，根据多元模型结合预测结果。首先进行数据预处理包括**数据集扩充、数据集划分，数据标注**等任务；其次，在建立好的数据集上，采用**迁移学习**的的方法选取在大规模数据集上预训练的权重，应用到 VisionTransfromer **模型**对图像特征提取，分析特征并进行第一轮预测；采用 momentum **动量优化**方法达到学习率动态优化，利用梯度的移动指数加权平均使学习率最优；Dropout **算法**防止过拟合和的同时组合多元神经网络，最终用 k **折交叉验证算法**的准确性。通过多方面综合手段，模型预测结果极其接近于真实值，效果良好，为后续优化模型和评估模型提供了基础。

针对问题二，主要改进相关数据集的预测效果，评估预测的精度和速度，并检验预测的合理性。本文首先根据多次实验结果，在平衡训练速度、模型性能和硬件资源的情况下进行了**超参数调优。**其次，第二轮预测根据坑洼道路预测的“犹豫”现象**采用模型融合的思路**，利用**软投票加权**的方式打分后归一化处理，得到合理的预测结果。在评价指标的选择上，采用 F1 **等指标**量化模型预测效果，并利用 PR **曲线和** ROG **曲线**可视化模型的预测能力，最终将融合模型的真实效果**与常见传统网络模型比对**，在结果上证明了模型的高效性和准确性。

最后，本文对所建立的模型的进行了整体评价及推广。

关键词：坑洼道路识别，特征工程，Vision Transformer，集成学习

目录

一、问题重述 1
1.1 问题背景 1
1.2 问题提出 1
二、问题分析 2
2.1 问题整体分析 2
2.2 问题一的分析 3
2.3 问题二的分析 4
三、问题假设 4
四、符号说明 5
五、数据预处理 5
5.1 原始数据分析 5
5.2 数据集的扩充 7
5.2.1 高斯模糊 8
5.2.2 图像旋转 9
5.2.3 图像裁剪 9
5.3 数据集划分 9
5.4 数据集标注 10
六、针对问题一的模型建立与求解 10
6.1 基础坑洼分类网络的对比分析 10
6.2 图像预处理 13
6.3 特征提取分析 14
6.4 模型结构建立 15
6.4.1 网络结构 15
6.4.2 momentum 动量优化算法 16
6.4.3 Dropout 16
6.4.4 交叉验证 17
七、针对问题二的模型评估与分析 17

7.1 模型训练及其超参数调优 17
7.2 模型预测结果合理性分析 18
7.3 评价指标选择 20
7.4 常见分类模型效果对比评估 22
八、模型整体评价与推广 24
参考文献 25
附录 25

一、问题重述

1.1 问题背景

我国道路交通设施的日益完善，使得人们的出行更加便捷。然而，我国城市道路在长期的使用过程中，受到温度、湿度、风化等各种自然因素以及交通荷载的影响，容易出现坑洼、裂缝等损坏现象。坑洼问题对道路安全造成严重影响，给人们的生活带来诸多不便，如汽车爆胎、轮胎损坏、交通事故等。这些问题不仅缩短了道路的使用寿命，还提高了交通事故的风险。

因此，路面状况与我们的安全息息相关，对道路状况进行实时、有效的监测和评估，对于提高道路养护管理水平、保障交通安全具有重要意义。为解决这一问题，需要对路面状况进行持续监测和维护，以确保人们的出行安全。

坑洼道路的检测和识别任务作为计算机视觉领域的一个重要研究方向，旨在实现对道路表面状况的自动检测、识别和分类。由于不同坑洼图像的之间特征差异性、环境光照强度以及道路过往车辆的遮挡问题，传统的分类算法往往不能取得很好的分类识别效果。

近年来，随着深度学习的发展，计算机视觉领域的识别、检测和分割问题都得到了很好的解决。相比机器学习研究，深度学习的神经网络可以自适应地提取物体相应的关键特征，因此将深度学习方法应用到道路坑洼检测识别领域之中，对未来的自动驾驶发展领域、地质勘探领域、自然灾害分析领域的研究和应用皆具有重要意义。

1.2 问题提出

本赛题希望通过对已标记的道路图像进行分析、特征提取与建模，从而能对于新的无标签道路图像进行自动识别坑洼状态，需解决问题如下：

问题 1：结合附件一内给出的图像文件数据，提取图像特征，建立一个识别率高、速度快、分类准确的模型，用于识别图像中的道路是正常或者坑洼。

问题 2：对问题 1 中构建的模型进行训练，并从不同维度进行模型评估。

问题 3：利用已训练的模型识别测试集中的坑洼图像，并将识别结果放在“test_result.csv”中。

二、问题分析

2.1 问题整体分析

目前，道路坑洼场景呈现多样化特点，包括无坑洼道路(即正常道路)、单坑洼道路以及多坑洼道路等。此外，天气条件、坑洼形状和深浅程度等因素为多场景坑洼分类研究及应用带来显著挑战。

第一，在无坑洼道路和单坑洼道路分类方面，技术实现相对较为简单。然而，随着道路状况复杂度的提高，多坑洼道路分类面临较大困难。在这种情况下，不仅需要准确识别和区分各坑洼，还需考虑其相互关联及动态变化。

第二，天气状况或光照环境对坑洼可见性和识别难度具有显著影响。在雨雪等恶劣天气条件下，拍摄照片数据内的坑洼可能被雨雪掩盖，增加识别难度；其次需识别的道路都在室外，在正午、黄昏及深夜等不同时段的外界环境光照影响较大且差异明显，对拍摄照片数据内的坑洼特征提取造成了一定程度的困难。

第三，不同类型的坑洼形状和深浅程度对分类算法提出了更高要求。例如，深而窄的坑洼可能容易被忽视，而浅而宽的坑洼则更容易被发现和分类。

为了应对上述挑战，问题解决可从以下几个方面展开：

在数据预处理角度，首先我们观察到附件 1 内所给训练数据为小样本数据集，通常训练一个识别准确率高的神经网络需要大量的数据，如在经典的 MNIST 手写数据集识别任务中共使用了 6 万张图片进行训练，才能将准确率控制在 90%以上，而对于本题目的坑洼道路识别任务中，所给数据集仅有 301 张数据图片，其中包括 266 张正常道路照片（下文称 normal 数据）以及 35 张坑洼道路照片（下文称 potholes 数据）。针对这种小样本数据，我们首先需要确定数据预处理方法，然后进行数据集扩充，在数据集扩充后进行相应数据集划分，并在数据集扩充过程中确定相应的异常处理方法，使数据预处理代码具有一定的鲁棒性和可靠性，从而达到优质的数据预处理需求，为后续的模型建立及评估打下良好坚实的基础。

在模型选择、建立及训练角度，首先我们需要进行模型的对比分析，确定相应的主干模型及附属模型，并仿用集成学习的思路，加权结合多种分类预测模型，共同采样提取坑洼道路有效特征。在建立高效可靠的模型结构后，进行模型的超参数调优，选取合适的模型参数，准确提取坑洼特征，在对 normal 数据和 potholes 数据的打分加权投票后，得出合理的预测结果，并对模型预测结果进行合理性分析。

在模型评估角度，选取合适的评价指标，绘制便于数据分析的评价曲线，从不同维度进行模型评估和校验。

2.2 问题一的分析

问题一要求我们结合给定的图像数据，提取图像特征并建立一个识别率高、速度快、分类准确的模型，用于识别图像中的道路是正常还是坑洼。在问题一的分析中，我们将关注两方面：图像特征提取方法的选择设计和深度学习网络模型结构的建立。

图像特征提取方法的选择和设计方面，由于目前深度学习领域的飞速发展，目前主流的特征提取网络多种多样，各自有其特点和适合处理的问题，我们将通过对附件 1 中所给的 301 张图像进行初步的探索性分析，初步选取一些坑洼识别领域的主干网络用来进行图像特征提取。

初步选定特征提取方法后，我们根据数据集的规模，问题的复杂度等进行试验，设计合适的用来图像分类的深度学习网络的结构。模型大体要包括：数据预处理、特征提取器、模型分类器三部分。

在数据预处理部分，为了满足算法输入格式的要求我们需要对图像尺寸等进行调整，为了满足任务对模型泛化能力的要求，我们需要对数据进行旋转、模糊、剪裁、等增强处理，对数据集进行增广，从而扩充训练集的数量和多样性；特征提取部分，我们需要根据试验结果对选取的网络模型结构及参数进行一些调整，同时为防止过拟合需采取一些正则化手段；模型分类器部分，如果我们提取到的特征足够有效，一般都能取得好的分类结果，但仍可以考虑针对这个任务进行一些优化，比如对数据不均衡的问题对损失函数进行调整。问题一的解决方案事件流图如下图所示。

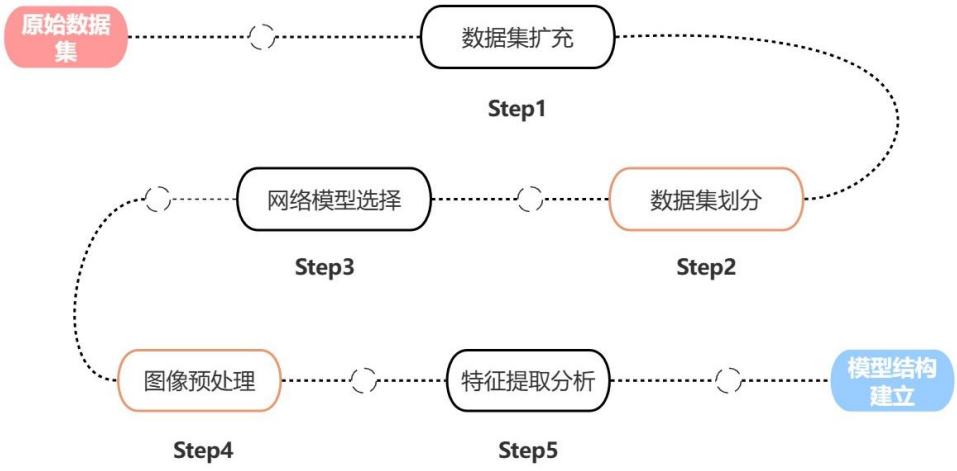


图 2-1 问题一解决方案事件流图

2.3 问题二的分析

问题二要求我们对问题一中构建好的模型结构进行训练，并从不同维度进行模型评估。问题二解决的先决条件是，首先进行模型训练和超参数调优，在取得优质的模型训练结果后，对各层训练结果进行模型保存，撰写相关评估代码及坑洼道路的模型预测分类代码。

在问题二的分析中，我们将关注以下两部分内容：模型评价指标的选取以及模型效果的相应指标评估分析。

对模型评估常见指标进行选取时，可以考虑如准确率、精确率、召回率、F1 分数等评价指标以及 P-R 曲线、ROC 曲线及 AUC 等评估图像的绘制。完成评估代码撰写以及相应图像绘制后，进行相应评估曲线图像趋势判别分析和评估指标数值分析，同时应对模型在不同维度上进行综合评估，如时间效率、鲁棒性等。问题二的解决方案事件流图如下图所示。

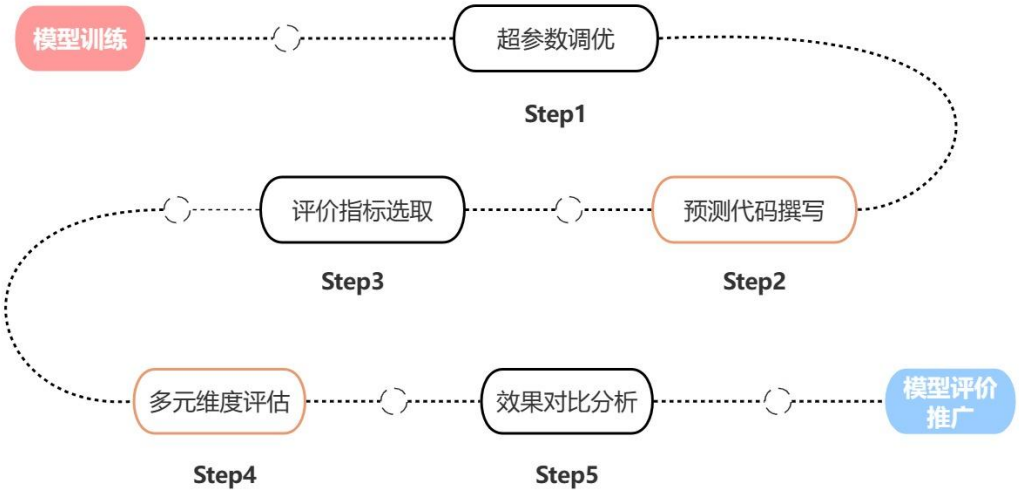


图 2-1 问题二解决方案事件流图

三、问题假设

我们将基于以下假设进行分析和建模：

1. 假设附件 1 给定的数据集内可全面体现正常道路及坑洼道路特征信息；
2. 附件 1 数据集内的图像标记准确率较高，可以满足模型建立和有效训练的需求；
3. 对于坑洼数据集内的 potholes 数据，内在特征规律具有相关性。

四、符号说明

表 4-1 符号说明

符号	符号说明
W	图像宽度
H	图像高度
C	网络模型的通道数
y	数据实际标签(0 表示坑洼，1 表示正常道路)
$\hat{y}$	模型的预测概率
train	训练集符号
val	验证集符号
test	测试集符号

五、数据预处理

5.1 原始数据分析

当前，研究坑洼道路的多种场景涉及到各种复杂情况，包括但不限于不同道路上的有坑洼和无坑洼的情况，以及单一道路上存在的单个坑洼和多个坑洼的不同情形。同时，还需要考虑到天气状况、坑洼的形状和深度等因素，这为坑洼多场景分类的研究和应用带来了巨大挑战。在题目给出的原始数据集中，共 301 张图片数据，属于小样本数据集。其中包括正常道路数据（下称 normal 数据）共 266 张，坑洼道路数据（下称 potholes 数据）共 35 张。

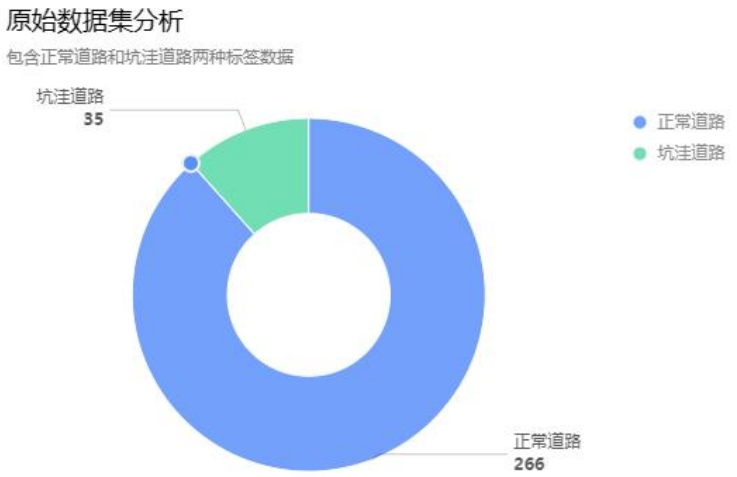


图 5-1 原始数据集统计图

通常训练一个识别准确率高的神经网络需要大量的数据，如在经典的 MNIST 手写数据集识别任务中共使用了 6 万张图片进行训练，才能将准确率控制在 90%以上。而如上图所示，原始数据集内仅包含 301 张图片数据，这种小样本数据集可能导致模型的性能下降或者难以充分发挥深度学习的优势，小样本数据集的模型构建难点如下：

其一，过拟合风险增加。在小样本数据集中，模型更容易记住训练数据而不是学习数据的潜在模式。因此，小样本数据集更容易导致过拟合，即模型在训练集上表现良好，但在新数据上表现较差。

其二，泛化性能不佳。深度学习模型通常需要大量的数据来学习数据的抽象特征和模式。在小样本数据集中，由于数据量不足，模型可能无法捕捉到数据的真实分布，从而导致泛化性能不佳，即在未见过的数据上无法准确预测。

其三，梯度下降不稳定。深度学习模型通常使用梯度下降等优化算法来更新参数，使得损失函数最小化。在小样本数据集中，由于数据量少，梯度的估计可能具有较大的方差，导致优化过程不稳定，难以找到全局最优解。

若想使用题目所给的原始数据集完成模型建立并训练出优质的模型分类效果，必须要设计合理的方法进行模型构建，解决方案主要分为如下三点，解决方案图如下所示。

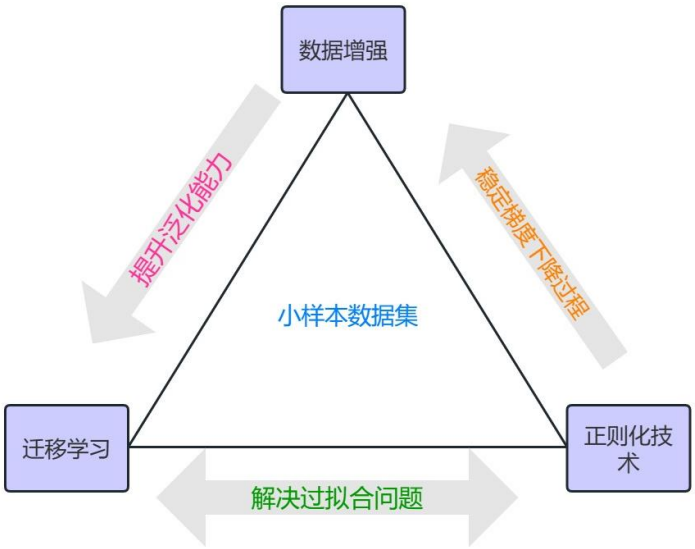


图 5-2 小样本数据集解决思路图

第一，数据增强。即通过对现有数据进行旋转、翻转、缩放等操作，生成新的样本，增加数据量，有助于提高模型的泛化性能。

第二，迁移学习技术。利用在大规模数据集上预训练的模型，将其部分或全部网络结构迁移到小样本数据集上，加速模型训练并提高性能。

第三，正则化技术。使用正则化技术（如权重衰减、Dropout）来减少过拟合风险。

5.2 数据集的扩充

原始数据集内的正常道路部分包含了如图 5-3 内所示的情形，其中 a、b、c、d、e、f、g、h 分别代表正常道路数据下的强光下晴天、弱光下晴天、强光下雪天、弱光下阴天、亮灯的夜晚、弱光下雪天、正常光雾天、隧道内光线共八种情况。



(a) (b) (c) (d)



(e) (f) (g) (h)

图 5-3 正常道路数据的各种情形

原始数据集内的坑洼道路部分包含了如下图所示的几种情形，其中 a、b、c、d、e 分别代表坑洼道路数据下的晴天、雨天、雪天、有积水、多坑洼和车辆遮挡共五种情况。



(a) (b) (c)



(d) (e) (f)

图 5-4 坑洼道路数据的各种情形

针对以上各种情形，分别进行数据集扩充，但又保证不影响原始数据集，故采用了图像处理技术进行数据增广即数据集扩充，如高斯模糊、图像旋转、图像剪裁等数据集扩充方式，从而保证原始数据集特征一致性。

5.2.1 高斯模糊

高斯模糊（Gaussian blur）是一种常用的图像处理技术，用于降低图像的噪声、平滑图像、以及在某些情况下提供边缘保持。该模糊方法的核心思想是使用高斯函数来对图像进行加权平均，以达到模糊图像的目的。

高斯模糊的原理基于高斯分布函数，也称为正态分布。高斯分布函数表示为：

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

其中，*G(x,y)*是二维高斯函数在坐标点*(x,y)*处的值，*σ*是标准差，用于控制高斯分布的宽度，标准差越大，模糊效果越明显。公式中的 e 是自然对数的底（约等于 2.71828）。

在图像处理中，高斯模糊的操作是将图像中每个像素点的值，根据周围像素的权重进行加权平均。权重由高斯函数决定，离目标像素越远的像素点权重越小，离得越近的像素点权重越大。通过这种方式，图像的细节被模糊化，达到降噪和平滑的效果。高斯模糊的公式如下：

$$I_{\text{blurred}}(x,y) = \frac{1}{\sum_{i=-k}^{k}\sum_{j=-k}^{k} G(i,j)} \sum_{i=-k}^{k}\sum_{j=-k}^{k} G(i,j) \times I(x+i, y+j)$$

其中，$I_{blurred}(x,y)$是模糊后图像在坐标点*(x,y)*处的像素值，*I(x+i,y+j)*是原始图像在*(x+i,y+j)*处的像素值，*G(i,j)*是高斯函数在*(i,j)*处的值。*k* 通常是高斯核的大小，控制了模糊的程度。较大的 *k* 会导致更强的模糊效果。

取高斯核大小 *k* 分别为 25、35、45、55，对均匀分成四份的 normal 数据进行高斯模糊处理，并对整体 potholes 数据分别进行一次高斯模糊处理。处理后生成 normal 数据 266 张，potholes 数据 140 张，生成相关数据样式如下例。

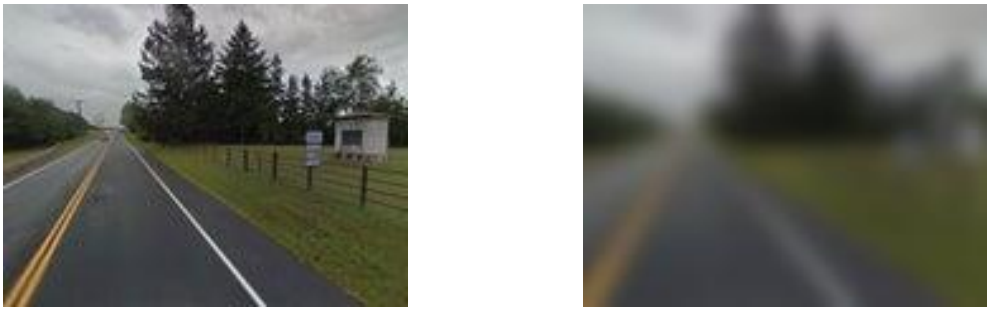


图 5-5 高斯模糊处理后的数据效果

5.2.2 图像旋转

图像旋转是扩充数据集的一种常见的方式，即把图像中的像素都按照某一个方向都旋转一定的角度，从而把目标原有的位置以及方向进行相应的改变。

取旋转角度θ为-180°到 180°之间的随机值，对四分之一的 normal 数据随机采样进行图像旋转处理，并对整体 potholes 数据分别进行一次处理。处理后生成 normal 数据 67 张，potholes 数据 35 张，生成相关数据样式如下例。



图 5-5 图像旋转处理后的数据效果

5.2.3 图像裁剪

裁剪在数据增广中也是一种常见的方式。有时，因坑洼道路图像中的坑洼特征有时位于图像下侧或右侧，故选取图像剪裁方式进行数据集扩充，取前景或中景裁剪。对四分之一的 normal 数据随机采样进行裁剪处理，并对整体 potholes 数据分别进行一次处理。处理后生成 normal 数据 67 张，potholes 数据 35 张，生成相关数据样式如下例。



图 5-6 图像裁剪处理后的数据效果

5.3 数据集划分

在对原始数据数据集扩充后，得到扩充后 normal 数据共 666 张，potholes 数据共 245 张，并将训练集和验证集随机划分 8:2 的比例，用于模型选取实验。数据集详情如下。

表 5-1 数据集划分表格

数据分布						
原始数据	图像模糊	图像旋转	图像裁剪	数据总量	训练集	测试集
301	406	102	102	911	729	182

5.4 数据集标注

数据标注是深度学习最基础的工作，同时也是最重要的一个环节。经题目需求统一将将正常道路数据标记为 1，将坑洼道路标记为 0。

六、针对问题一的模型建立与求解

6.1 基础坑洼分类网络的对比分析

分类网络的分类效果取决于神经网络学习到的图像特征的好与坏。因此搭建一个效果不错的特征提取网络很有必要。在对 VGGNet、RESNet、VisionTransformer 三种网络的对比分析后，选取合适的特征提取网络，并通过训练调参，得到能够提取相应坑洼特征的网络权重值，最后经过分类器，可以实现不同场景下正常道路与坑洼道路的分类。

本文将上述三种网络模型进行介绍，并对比分析如下。

（1）VGGNet 网络

VGGNet 是一种经典的卷积神经网络架构，它是由 16 层权重矩阵构成的神经网络，叠放了多个 3×3 的卷积层、2×2 池化层和 ReLU 激活函数的基本块(block)构成。在最后，还有两个全连接层，它们将卷积神经网络的输出转换为一组分类输出。

首先，VGGNet 采用了较深的网络结构，通过多个卷积层和池化层的堆叠来提取图像特征。这种设计使其能够对图像进行更准确的分类和特征提取，但也导致了较大的模型参数和计算量。其次，VGGNet 使用较小的卷积核（通常是 3x3 大小），这样可以增加网络的非线性表示能力，并减少模型中的参数数量。同时，VGGNet 在网络的设计中采用了参数共享的策略，即多个卷积层使用相同的权重，这样可以减少模型的参数量，但也限制了模型的灵活性。该网络的模型结构如下图：

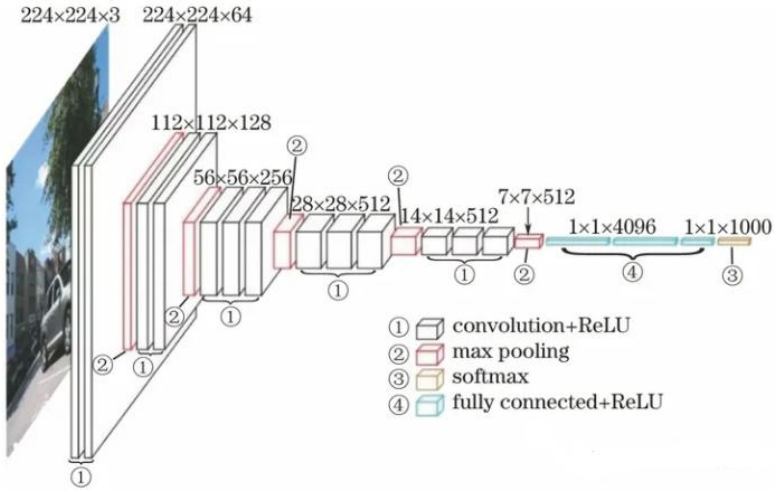


图 6-1 VGG16 网络模型结构图

（2）RESNet 网络

RESNet 是一种具有残差连接的卷积神经网络架构，它的关键特点是引入了跨层连接，解决了深层网络训练过程中的梯度消失和模型退化问题。

RESNet 中的每个残差块包含跨层连接，允许梯度在网络中直接传播，减轻了梯度消失问题，使得网络更易训练。

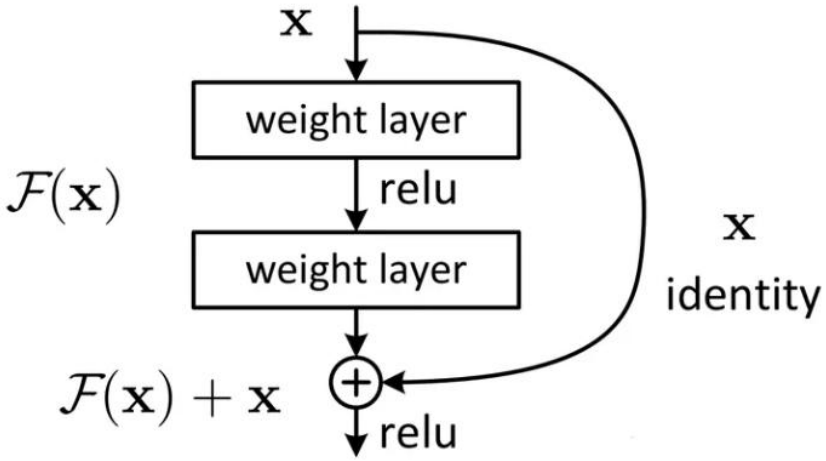


图 6-2 残差块结构图

这种设计使得 RESNet 可以构建非常深的网络，有助于提取更复杂的特征。RESNet 可以通过堆叠多个残差块来构建深层网络，从而提高网络的表示能力。较浅层的特征可以通过跨层连接直接传递到后续层，为网络提供了更多的信息。

（3）VisionTransformer(ViT)

VisionTransformer 是一种基于自注意力机制的图像分类模型[1]，它将图像分割成小的图块，并使用自注意力机制来捕捉图块之间的依赖关系。这使得模型可以在不依赖于固定的卷积操作的情况下，有效地捕捉长距离的依赖关系，从而提高了模型的表示能力。VisionTransformer 将输入图像分割成小的图块，并使用嵌入层将每个图块映射到低维向量表示。这种设计使得模型可以处理任意大小的输入图像，而不受固定大小的卷积操作的限制[2]。

下面进行网络结构理论对比分析，首先，RESNet50 的参数数目明显比 VGGNet 更少，因此在参数数量受限制的应用中，RESNet50 可能更容易被部署到实际任务中。但是，由于 RESNet50 中跨捷径的加入，使得模型更加深度、复杂，同时在训练过程中需要较高的计算资源，因此训练复杂度会增高。而 VGG16 相对于 RESNet50 的优点在于模型的稳定性更好，容易收敛，它只包含非常多的卷积层和全连接层，适合中小型数据集的图像识别任务。相比之下，VisionTransformer 引入了自注意力机制，可以在不依赖

固定卷积操作的情况下，有效地捕捉图像中的全局依赖关系。传统网络通常使用卷积层来提取局部特征，并可能存在感受野较小的限制。VisionTransformer 还在许多图像分类任务中展现出色的推广性能，VisionTransformer 具有较强的泛化能力和适应性。此外，VisionTransformer 的设计和应用仍在不断演进和改进中，因此在特定任务和数据集上的性能可能会有所差异。

本文经过之前的理论分析后，开始进行网络模型的实验对比分析。其中，实验数据为扩充后的坑洼道路数据集，同时选取轮次 epoch=20，批大小 batch_size=6，初始学习率为 0.001 且进行动量优化学习率设置，进行网络模型训练，得到具体性能对比数据如下表，其中 Loss 值为损失。

表 6-1 各网络实验性能对比

算法模型	数据集	训练速度（每个 epoch）	平均准确率	最佳准确率
VGG16	坑洼数据集	10s	0.945	0.961
RESNet50	坑洼数据集	6s	0.961	0.978
ViT	坑洼数据集	5s	0.994	1.0

根据实验结果，VGGNet 在道路坑洼分类任务上取得了较高的准确率，但训练速度相对较慢。由于网络较深且参数较多，VGGNet 在小数据集上可能容易出现过拟合的情况。

相比 VGGNet，RESNet 在道路坑洼分类任务上具有更高的平均准确率和最佳准确率。由于使用了残差连接，RESNet 克服了传统深层网络的问题，即在训练过程中难以训练和优化。它能够更好地捕捉道路坑洼的特征，并在小数据集上取得较好的性能。

但对比其余两类模型，VisionTransformer 在道路坑洼分类任务上表现出色，具有非常高的平均准确率和最佳准确率。由于自注意力机制的引入，VisionTransformer 能够更好地捕捉图像中的全局依赖关系，并在小数据集上取得非常高的性能。同时，VisionTransformer 的训练速度稍微也明显比其他模型更快，同时兼顾了训练速度和结果准确度。

6.2 图像预处理

数据集内的图像信息在进入神经网络之前，由于采集到的道路图像的大小尺寸不统一，因此应将数据集大小尺寸处理一致。尺寸的统一对分类网络的效果有一定的影响，尽可能的使数据集的图像的大小的尺寸一致，因此采用了 resize 的方式，将多场景坑洼分类的数据集的图像的大小统一为 256*256，然后再进行网络模型训练。

针对于典型的坑洞图像，具有以下的易观测特征：坑洼通常是凹洞、坑洼纹理通常比周围的路面更加粗糙和颗粒化。由于阴影，坑洼 ROI 像素的强度通常比周围路面的强度，但是由于坑洼中常常存在污水、拍照角度和光线不合适、坑洼位置偏僻等原因，整幅图像色泽较暗，坑洼不易被发现，因此对调整大小后的图像进行图像对度增强。

直方图均衡化是一种常用的图像增强技术，它可以通过重新分配图像像素值的灰度分布来增强图像的对比度和细节。直方图均衡化可以通过扩展图像的灰度范围，使得原本较暗或较亮的区域得到更好的展示。这样可以增加图像中的细节和对比度，使得图像更加清晰和易于分析。尤其在灰度级分布不均匀的图像中，直方图均衡化能够显著改善图像的视觉效果。其次，直方图均衡化可以增强图像中的细节信息。通过重新分配像素值，原本相近的灰度级之间的差异被增大，从而使得细节更加突出，适用于坑洼检测任务。

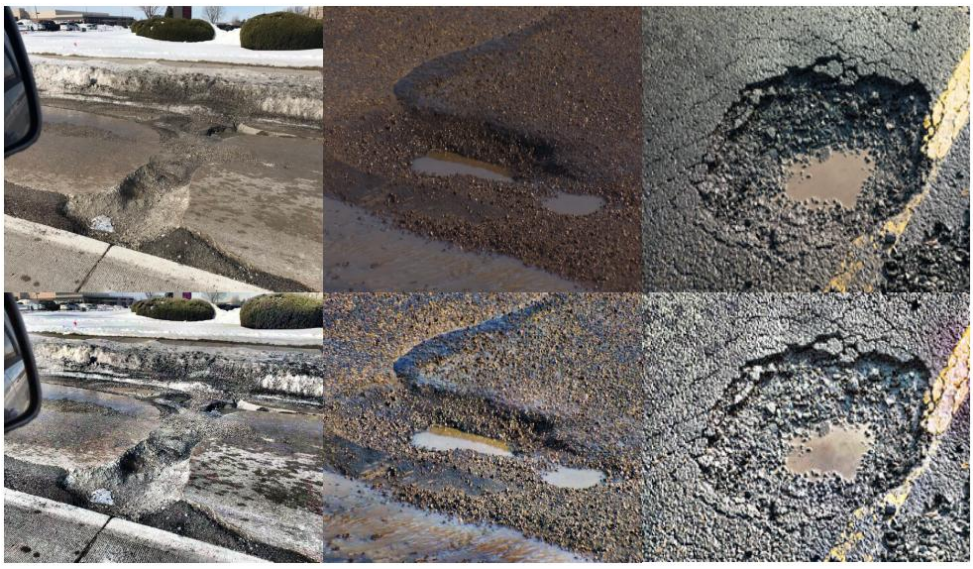


图 6-3 对比度增强前后坑洼图

由上图可见，直方图均衡化可以改善图像的视觉效果，使得图像更加饱满和生动。通过增强图像的动态范围，原本过暗或过亮的区域可以得到适当的平衡。

6.3 特征提取分析

ViT 通过将图像分成一系列的图块（patches），并将每个图块转换为向量表示作为输入序列。然后，这些向量将通过多层的 Transformer 编码器进行处理，其中包含了自注意力机制和前馈神经网络层。这样可以捕捉到图像中不同位置的上下文依赖关系。最后，通过对 Transformer 编码器输出进行分类或回归，可以完成特定的视觉任务[3]。以下为本文模型特征提取步骤：

第一步，输入一张尺寸为 h*w*c 的图像，h，w 的值均为 224;

第二步，将图像分成 p*p*c 的小块，那么一共可以获得 n 个图像块, n=(h*w)/(p*p)，同时添加一个可学习的类别块，则总共有 n+1 个块待处理，这个类别块将用于与所有的图像小块进行交互，最终从类别块中学习到用于分类的特征；

第三步，对图像小块进行 flatten 操作，即将每个图像块 p*p*c 转换成 1*(p*p*c) 大小的一维向量，再将 n 个一维向量拼接起来，组成 n*(p*p*c) 的二维向量。然后再对二维向量使用全连接层进行降维得到 n*1024 的二维特征 a。

第四步，对输入特征加入位置编码。位置编码用于标示每一个图像块的相对位置。

第五步，将预处理特征送入 transformer encoder 中得到交互特征 f。在 encoder 中最重要的就是多头注意力层，对于输入特征 a (n*1024) ，将其划分为 m 个头，得到 m 个不同的特征 (a1,a2……am) 。例如有 12 个头，则对于某一个输入特征 a(n*1024) ，就会划分出 12 个不同的特征 (a1,a2……a12),尺寸为 n*(1024/12) ，然后在这 12 个特征中执行自注意力计算如图 6-4 所示，得到 12 个加权特征 (b1，..., b12)，再将其拼接到一起得到 z(n*1024) ，通过非线性变化 w，最终得到与输入特征同尺寸的交互特征 f(n*1024) ，f=w*z, 其中 w 的维度为 n*n,Z 的维度为 (n*1024)。

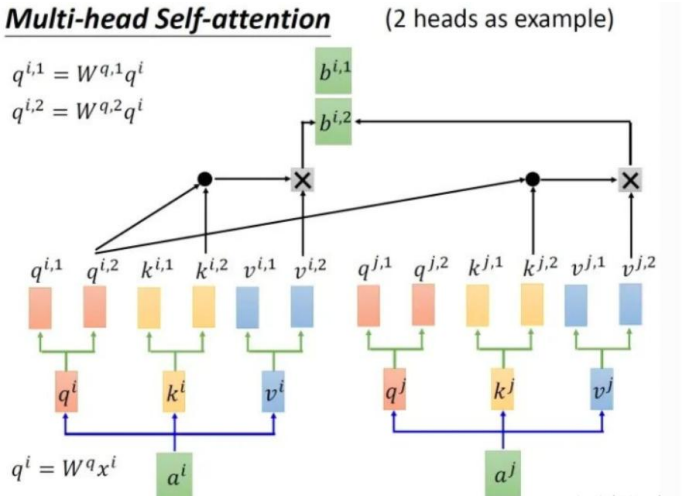


图 6-4 多头自注意力计算过程

第六步，对于经 encoder 得到的交互特征 f(n*1024)从中只抽取代表类别块的一维特征 (1*1024)，用作后续的分类。再通过 MLP 进行降维操作，将 1024 降维为待分类的类别数，即 (1* class_num)，此处 class_num=2,即可完成分类。坑洼图像提取特征示意图如下：

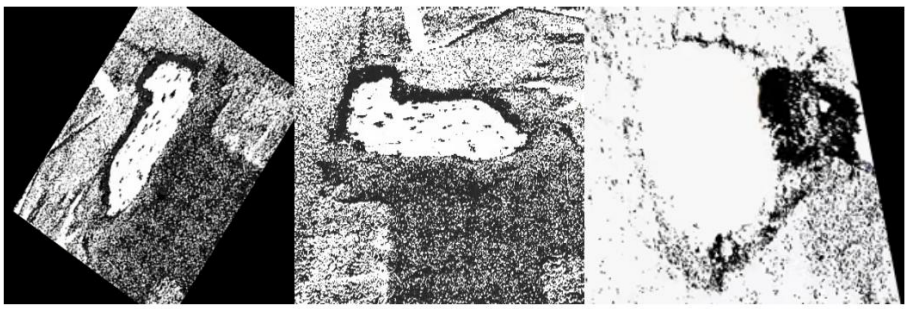


图 6-5 坑挖图像特征提取示意图

可以看出，坑洼部分被白色加亮处理，与周边正常马路区域变化明显，噪音更少更突出一些，为后续预测分类结果做了充足的准备。

6.4 模型结构建立

6.4.1 网络结构

VisionTransformer 模型将输入图片分成多个 patch（16*16）再将每个 patch 投影为固定长度的向量送入 Transformer，后续 encoder 的操作和原始 Transformer 中完全相同。但是因为对图片分类，因此在输入序列中加入一个特殊的 token，该 token 对应的输出即为最后的类别预测。其网络模型结构图如下：

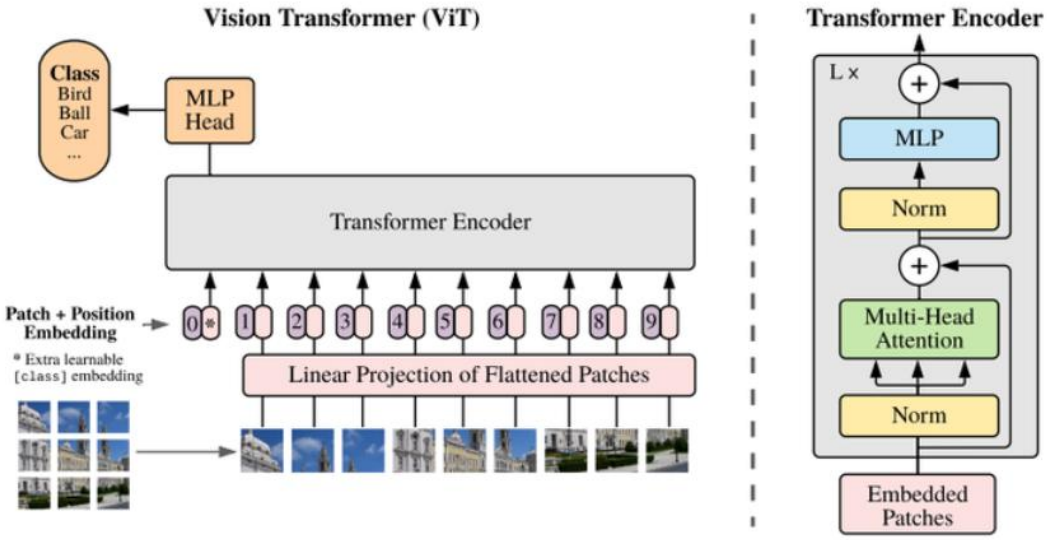


图 6-6 网络模型结构

另外，再模型建立的过程中，采用迁移学习的方法选取在大规模数据集上预训练的模型，将其网络结构迁移到小样本数据集上，加速模型训练并提高性能。本网络选用 vit_base_patch16_224_in21k.pth 预训练模型，在 resize 进行预处理后，通过 CenterCrop 方法（即去掉外侧边缘部分，留取中间区域内容），将图像裁剪为 224*224 大小的图像数据，送进网络模型。

6.4.2 momentum 动量优化算法

采用 momentum 动量优化方法达到学习率动态优化，由于训练深度神经网络的时候是每一批次的训练，要使得训练的结果达到最优点，且也不因为学习率低而收敛太慢，同时也不因为学习率过大，而使得摆动幅度过大，从而不能得到最优的效果，而 momentum 优化器可以解决上述的问题，其采用的是基于梯度的移动指数加权平均。momentum 优化算法即如下公式。

$$v_{dw} = \beta v_{dw} + (1-\beta)dW$$

$$v_{db} = \beta v_{db} + (1-\beta)db$$

$$W = W - \alpha v_{dw}$$

$$b = b - \alpha v_{db}$$

V_{dw} 和 V_{db} 分别代表在前一轮迭代过程中累积的权重和偏置的梯度动量，β是梯度累积指数，dW 和 db 分别是损失函数反向传播得到的权重梯度和偏置梯度，α是学习率。

6.4.3 Dropout

模型组合能够提高机器学习的性能。然而，对于大型神经网络，使多次独立训练网络的结果平均化的方法的代价是很高的。训练许多不同的体系结构是很困难的，因为找到每个网络架构的最佳超参数是一项艰巨的任务，训练每个大型网络需要大量的计算。此外，大型网络通常需要大量训练数据。

Dropout 就是处理上述问题的一种技术，它可以防止过拟合和同时能提供一种将许多不同神经网络组合的方法。dropout 就是删除神经网络的某些单元。通过删除一个单位，即从网络暂时删除它，包括与这个单元的所有传入和传出连接。要删除某个单元是随机选择的。在最简单的情况下，每个单元都要以独立于其他单元的概率 p 保留，其在每个批次数据训练的过程中，通过按概率 p 将隐藏层的某些神经元置为 0，这样可以缓解隐藏层神经元之间的相互作用，从而减少模型的过拟合现象。

6.4.4 交叉验证

在完成网络模型选取后，对于数据集的划分，应采用交叉验证的方法。交叉验证的基本思想是将原始数据进行分组，一部分做为训练集来训练模型，另一部分做为测试集来评价模型。*k* 折交叉验证一般步骤为将数据集分成 *k* 份，轮流将其中 *k-1* 份作为训练数据，*1* 份作为测试数据，进行试验。每次试验都会得出相应的准确率。*k* 次的结果的准确率的平均值作为对算法精度的估计，一般还需要进行多次交叉验证，再求其均值，作为对算法准确性的估计。

相比较一般划分训练集和测试集的方法，交叉验证的优点表现在：

（1）交叉验证用于评估模型的预测性能，尤其是训练好的模型在新数据上的表现，可以在一定程度上减小过拟合。

（2）可以从有限的数据中获取尽可能多的有效信息。

七、针对问题二的模型评估与分析

7.1 模型训练及其超参数调优

在完成学习率动态优化算法设计、Dropout 层设计、k 折交叉验证设计后，进行超参数的选择及调优。

当一个完整的数据集通过了神经网络一次并且返回了一次，这个过程称为一次 epoch，即所有训练样本在神经网络中都进行了一次正向传播和一次反向传播。然而，当一个 epoch 的样本，即所有的训练样本对于计算机而言的数量太过庞大时，就需要把它分成多个小块，也就是就是分成多个 batch 来进行训练。

在没有使用 Batch Size 之前，这意味着网络在训练时，是一次把所有的数据输入网络中，然后计算它们的梯度进行反向传播，由于在计算梯度时使用了整个数据库，所以计算得到的梯度方向更为准确。但在这情况下，计算得到不同梯度值差别巨大，导致网络的训练效果不好。

因此，分析并选择合适的 batch_size 和 epoch 是为了平衡训练速度、模型性能和硬件资源的使用，以便得到既高效又准确的深度学习模型。选择合适的 batch_size 和 epoch 通常需要进行实验和交叉验证，以找到最佳的组合。

下面实验以 ViT 模型为主干网络，初始设置学习率为 0.001，并采用 momentum 动量优化算法进行学习率调整。同时设置初始 epoch 为 15，batch_size 分别设置为 3、4、

5、6、10、20 六种。

可以观察到，在 epoch 为 10 时，模型已达到初步拟合，同时 batch_size 设置为 5 时，模型的损失函数下降速度快，且平均 loss 损失较低，可达到最佳效果。具体实验过程中，相关 loss 下降曲线如下图所示。

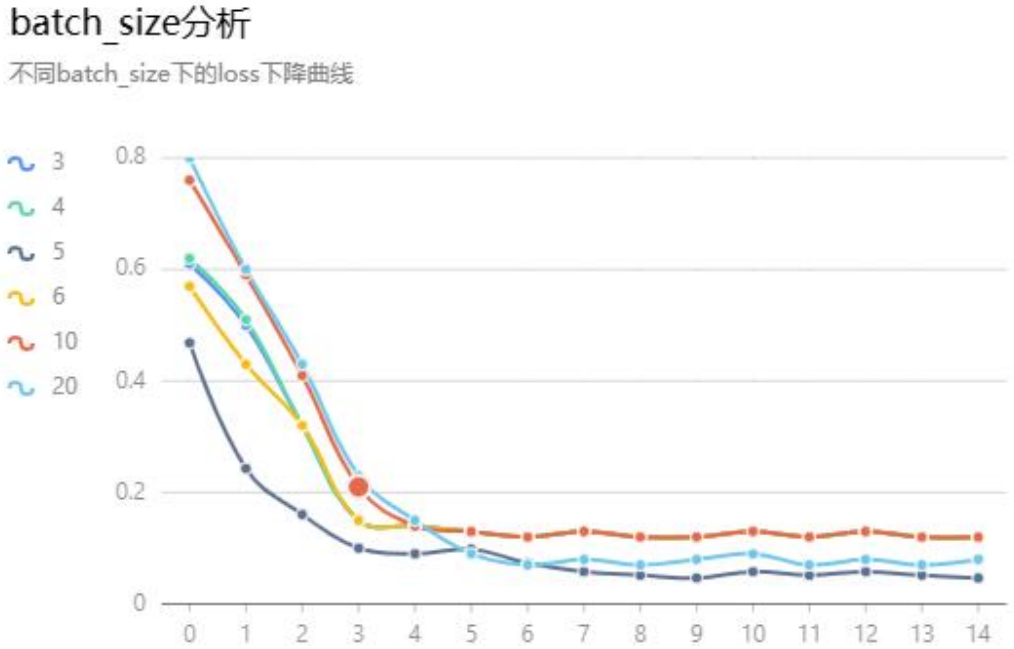


图 7-1 不同超参数下模型的损失函数曲线

故最终选取模型超参数如下表。

表 7-1 模型超参数选择表

超参数设置	参数值
分类数目	2
训练轮次 epochs	10
批量大小 batch_size	5
初始学习率 lr	0.001
学习率调整方法	momentum 动量优化算法

7.2 模型预测结果合理性分析

采用以上章节所述相关参数，在对 VGG16 模型、RESNet50 模型及 ViT 三类模型的训练以后，得到模型的平均准确率效果均大于 90%。而模型预测过程中，部分模型会对图像识别具有“犹豫”现象，即对 normal 的识别概率和 potholes 的识别概率都小于 0.7，称之为“犹豫”现象。故在坑洼道路预测过程中，对具有“犹豫”现象的识别图像，采

用模型融合的思路将三个模型的进行融合分类识别模型设计，使其共同对无标签的道路数据图像实现融合验证，采用软投票加权的方式如下。

$$result = \alpha \bullet result_1 + \beta \bullet result_2 + \gamma \bullet result_3$$

其中，α、β、γ分别为权重系数，*result* 表示为最终图像分类识别的概率结果，带有角标的 $result_1$、$result_2$、$result_3$ 分别为 VGG16 模型、RESNet50 模型及 ViT 三类模型的对应标签打分，然后将计算得出的 *result* 进行归一化处理，得到最终的分类概率。经过实验调试，最终确定权重系数组合为 $\alpha = 0.7$、$\beta = 0.9$、$\gamma = 1.2$。

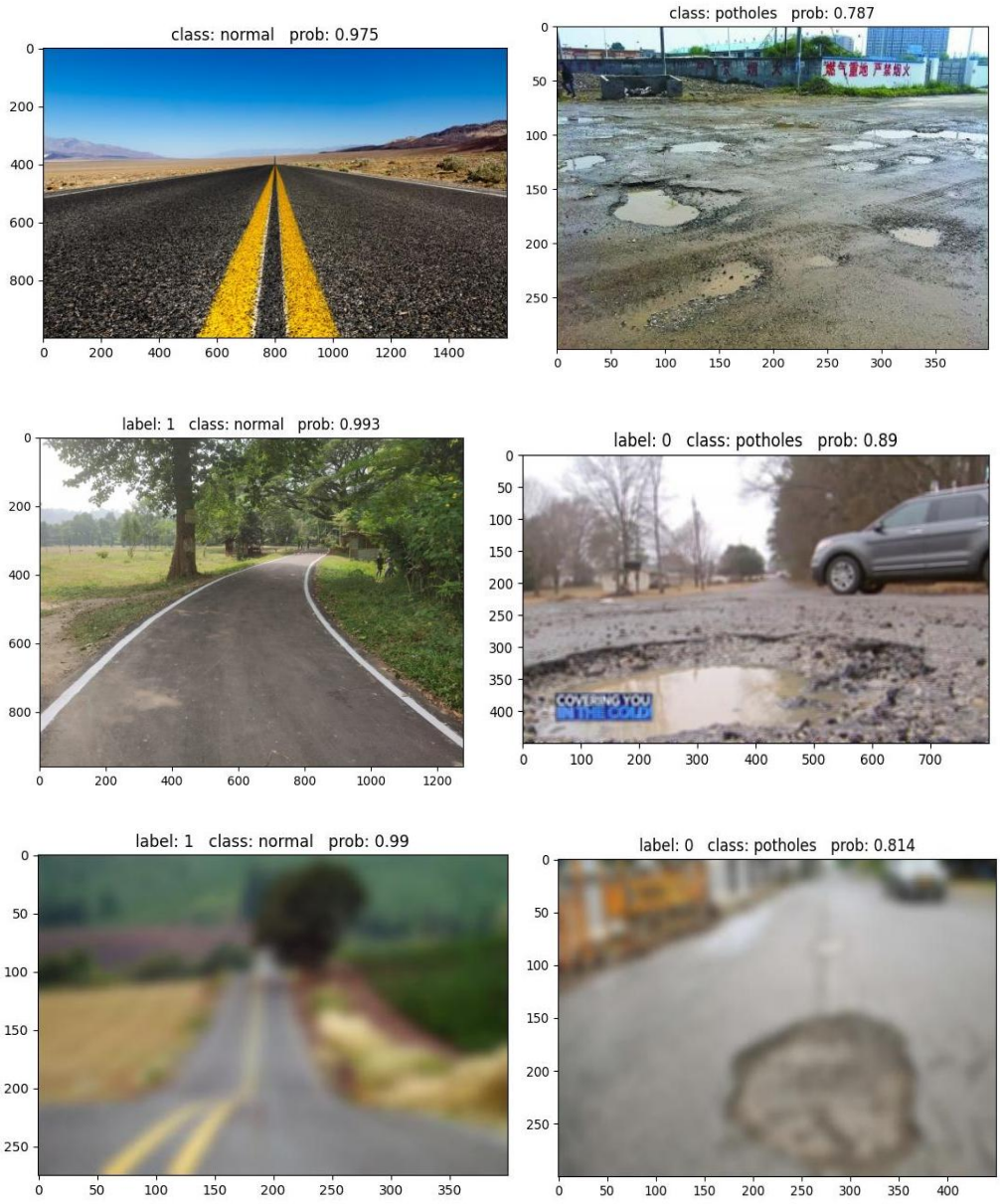


图 7-2 模型分类效果

经模型训练及模型验证后，得到最终模型预测的分类效果如上图，可以将左图正常

道路图像准确识别，分类权重为 0.975、0.993、0.99，也可以将右图坑洼道路图像准确识别，分类权重为 0.787、0.89、0.814，识别效果良好。

完成模型训练后，我们检索收集和题目所给原始数据集不重合的网络数据集进行模型效果测试，测试数据为 github 上的有标注坑洼数据集，共收集 normal 数据 352 张，potholes 数据 329 张，送入网络进行模型预测准确率验证。得到模型的混淆矩阵如下图所示。

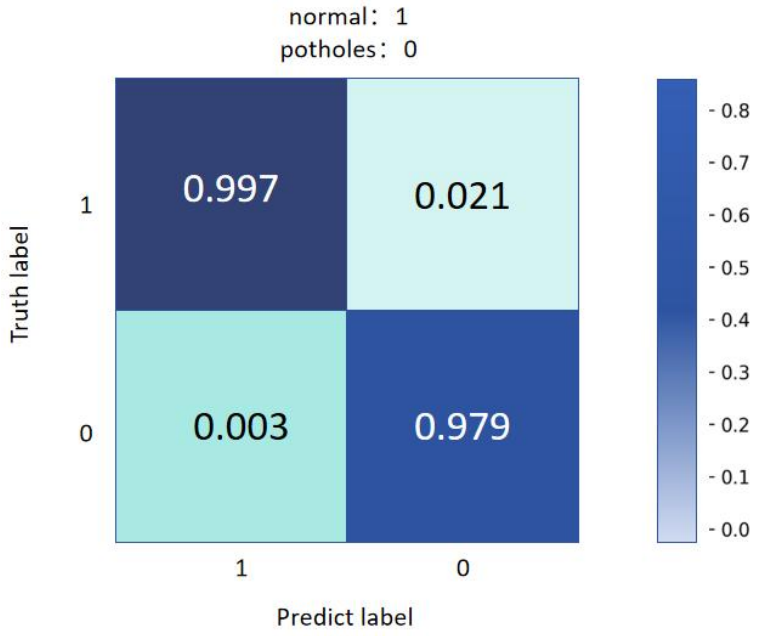


图 7-3 模型的混淆矩阵

且模型识别过程中，单张图片的识别速度如下表。

表 7-2 模型单张图片预测时间

数据集	网络模型	
	ViT 模型	融合模型
坑洼道路数据集	0.186s	0.521s

7.3 评价指标选择

图像分类 Q 的概念：是一个模式分类的问题，将不同的图像划分到不同的类别、单标签分类问题就是每一张图都有唯一的类别。

在计算这些指标之前，需要先计算几个基本的指标：

TP：正样本且分类为正样本的数目为 (True Positive)。

FN：标签为正样本分类为负样本的数目 (Flase Negative)。

FP：标签是负样本分类为正样本的数目 (Flase Positive)。

TN：标签是负样本且分类为负样本的数目 (True Negative)。

总结上述内容，如下表所示。

表 7-3 四类模型预测情况样式

	正样本	负样本
预测正例	TP	FP
预测反例	FN	TN

结合 TP、FP、FN、TN 四项情况数据，可以计算得出分类识别实验中常用的相关评价指标如下。

（1）Accuracy (准确率)

即对于给定的数据，分类正确的样本数占总样本数的比例。其计算公式如下：

$$Acc = \frac{TP + TN}{TP + FN + FP + TN}$$

（2）Precision (精确率)

分类正确的正样本个数占分类器所有的预测正样本个数的比例。即以判断为正例作为基准：模型判别为正例的里面，实际正确的概率是多少。其计算公式如下：

$$Precision = \frac{TP}{TP + FP}$$

（3）Recall (召回率)

分类正确的正样本个数占实际正样本个数的比例。以真实为正例作为基准: 真实值的正例中，被判断出来为正例的概率是多少。

召回率越高，精确率越低。例如概率阈值 T 的影响（概率阈值 T 是一个设定的在 [0,1] 范围的值，使用该值判断网络预测出的结果是正类还是负类)，当 T 小，导致召回率的升高，分类精确度下降，当 T 大，精确度升高，召回率降低。其计算公式如下：

$$Recall = \frac{TP}{TP + FN}$$

（4）F1-Score

即 F 值为正确率和召回率的调和平均值，当类别不均衡时可以作为比 Accuracy 更好的指标。其计算公式如下：

$$F_1 = \frac{2 \times precision \times recall}{precision + recall}$$

（5）ROC (Receiver Operating Characteristic)

准确率、精确率、召回率 和 F1-score 都是单一的数值指标，当需要观察分类算法在不同的参数下的表现，可以使用 ROC 曲线评价一个分类器在不同阈值下的表现情况。

在 ROC 曲线中，每个点的横坐标是 false positive rate(FPR)，纵坐标是 true positive rate(TPR)，描绘了分类器在 True Positive 和 False Positive 间的平衡，两个指标的计算如下。

FPN = FP / (FP + TN)，表示分类器预测的正类中实际负实例占所有负例的比例，FPR 越大，预测正类中的实际负类越多；

TPN = $TP/(TP+FN)$ ，表示分类器预测的正类中实际正实例占所有正例的比例。

另外，AUC 为 ROC 曲线下的面积，这个面积的数值不会大于 1。AUC=1 时，在任何阈值下分类器都可以 100% 识别所有类别，这是理想的分类器；$\mathrm{AUC}=0.5$ 时，相当于随机预测，此时分类器不可用，$0.5<\mathrm{AUC}<1$ 时，优于随机预测，这也是实际作用中大部分分类器所处的状态；AUC<0.5 时，总是比随机预测更差。AUC 作为一个评价标准，常和 ROC 曲线一起使用，可以看作是 ROC 的量化表现。

（6）Precision-Recall 曲线

Precision-Recall 曲线，又称 P-R 曲线。以精确率 (Precision) 为纵轴、召回率 (Recall) 为横轴作图，就得到了精确率-召回率曲线，简称 P-R 曲线。

PR 曲线功能说明，一条 PR 曲线要对应一个阈值（统计学的概率）。通过选择合适的阈值(比如 K%)对样本进行合理的划分，概率大于 K%的样本为正例，小于 K%的样本为负例，样本分类完成后计算相应的精准率和召回率，曲线越靠近右上角说明性能越好，可以根据曲线和坐标轴包围的面积来定量评估。AP 就是 Precision-Recall 曲线下围成的面积，通常来说一个越好的分类器，AP 值越高。

下节将结合上述评价指标，进行模型效果对比评估。

7.4 常见分类模型效果对比评估

我们检索收集和题目所给原始数据集不重合的网络数据集进行模型效果测试，测试数据为 github 上的有标注坑洼数据集，共收集 normal 数据 352 张，potholes 数据 329 张，送入网络进行模型预测准确率验证。

经 VGG16 模型、RESNet50 模型、ViT 模型、融合模型四类模型预测识别评估后，得到相关评价指标结果如下表。

表 7-4 相关评价指标情况

算法模型	精确率	召回率	F1 值	真阳性率	假阳性率	平均准确率
VGG16	0.931	0.943	0.937	0.943	0.07	0.937
RESNet50	0.954	0.967	0.960	0.967	0.047	0.960
ViT	0.967	0.991	0.979	0.991	0.037	0.994
融合模型	0.990	0.997	0.993	0.997	0.01	0.993

由上表及图 7-4、图 7-5 分析得知，融合模型在公开数据集上的最终分类效果达到了最佳，可知融合模型在相同分类模型下具有高识别率以及高分类准确率，达到了优质的效果。实验证明，本问题内建模的模型结构，可以在道路坑洼检测识别分类任务中，可以满足问题需求，具有良好的泛化能力和鲁棒性。

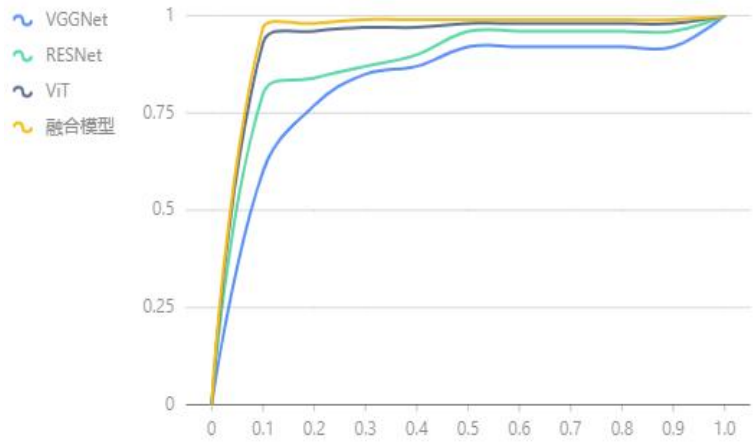


图 7-4 不同分类模型下的 ROC 曲线图

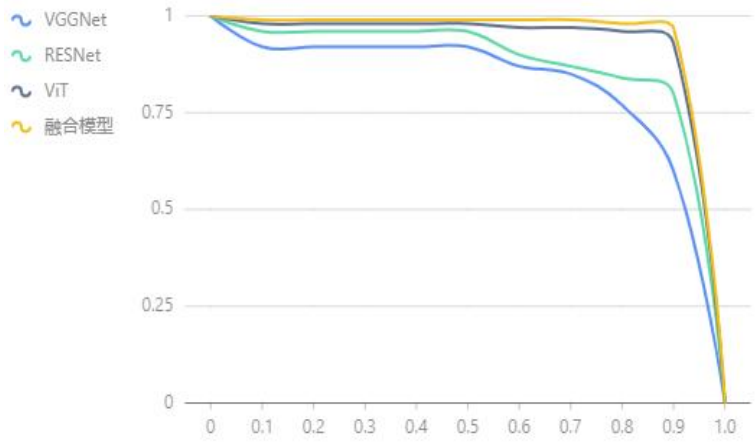


图 7-5 不同分类模型下的 P-R 曲线图

八、模型整体评价与推广

本文主要设计了基于坑洼表面特征的自动道路分类模型，较好的实现了对坑洼道路和正常道路的分类，在坑洼区域提取、道路安全检测和自动驾驶等方面的应用具有重大的意义。

本实验所建立的模型具有高度可扩展性，可以根据任务的要求进行灵活的调整和扩展。可以通过增加层数、隐藏单元数或头数来增加模型的容量，以适应更复杂的模式和更大规模的数据集。同时具有良好的泛化能力，即使在有限的数据情况下，本模型也能够对新的、未见过的数据进行较为准确的预测。

参考文献

[1] 张峰,黄仕鑫,花强等.基于 Depth-wise 卷积和视觉 Transformer 的图像分类模型[J].计算机科学 10(06):1-15,2023.

[2] 石争浩,李成建,周亮等.Transformer 驱动的图像分类研究进展[J].中国图象图形学报,28(09):2661-2692,2023.

[3]袁媛,陈明惠,柯舒婷等.基于集成卷积神经网络和 Vit 的眼底图像分类研究[J].中国激光, 49(20): 108-116, 2022.

附录

一、使用软件：

1） 文字与数学公式编辑：EduEditer

2） 流程图：Lucidchart

3） 编程：pycharm，vscode

4） 其他图表：Visio

二、关键代码

（1）model.py

```
# -*- coding: UTF-8 -*-
from functools import partial
from collections import OrderedDict
import os
import torch
import torch.nn as nn


def drop_path(x, drop_prob: float = 0., training: bool = False):
    """
    Drop paths (Stochastic Depth) per sample (when applied in main path of residual blocks).
    This is the same as the DropConnect impl I created for EfficientNet, etc networks, however,
```

the original name is misleading as 'Drop Connect' is a different form of dropout in a separate paper...

See discussion: https://github.com/tensorflow/tpu/issues/494#issuecomment-532968956 ... I've opted for

changing the layer and argument names to 'drop path' rather than mix DropConnect as a layer name and use

'survival rate' as the argument.

"""

if drop_prob == 0. or not training:

return x

keep_prob = 1 - drop_prob

shape = (x.shape[0],) + (1,) * (x.ndim - 1) # work with diff dim tensors, not just 2D ConvNets

random_tensor = keep_prob + torch.rand(shape, dtype=x.dtype, device=x.device)

random_tensor.floor_() # binarize

output = x.div(keep_prob) * random_tensor

return output

class DropPath(nn.Module):

"""

Drop paths (Stochastic Depth) per sample (when applied in main path of residual blocks).

"""

def __init__(self, drop_prob=None):

super(DropPath, self).__init__()

self.drop_prob = drop_prob

def forward(self, x):

```
        return drop_path(x, self.drop_prob, self.training)


class PatchEmbed(nn.Module):
    """
    2D Image to Patch Embedding
    """

    def __init__(self, img_size=224, patch_size=16, in_c=3, embed_dim=768, norm_layer=None):
        super().__init__()
        img_size = (img_size, img_size)
        patch_size = (patch_size, patch_size)
        self.img_size = img_size
        self.patch_size = patch_size
        self.grid_size = (img_size[0] // patch_size[0], img_size[1] // patch_size[1])
        self.num_patches = self.grid_size[0] * self.grid_size[1]

        self.proj = nn.Conv2d(in_c, embed_dim, kernel_size=patch_size, stride=patch_size)
        self.norm = norm_layer(embed_dim) if norm_layer else nn.Identity()

    def forward(self, x):
        B, C, H, W = x.shape
        assert H == self.img_size[0] and W == self.img_size[1], \
            f"Input image size ({H}*{W}) doesn't match model ({self.img_size[0]}*{self.img_size[1]})."

        # flatten: [B, C, H, W] -> [B, C, HW]
        # transpose: [B, C, HW] -> [B, HW, C]
        x = self.proj(x).flatten(2).transpose(1, 2)
```

```
        x = self.norm(x)
        return x


class Attention(nn.Module):
    def __init__(self,
                 dim,   # 输入 token 的 dim
                 num_heads=8,
                 qkv_bias=False,
                 qk_scale=None,
                 attn_drop_ratio=0.,
                 proj_drop_ratio=0.):
        super(Attention, self).__init__()
        self.num_heads = num_heads
        head_dim = dim // num_heads
        self.scale = qk_scale or head_dim ** -0.5
        self.qkv = nn.Linear(dim, dim * 3, bias=qkv_bias)
        self.attn_drop = nn.Dropout(attn_drop_ratio)
        self.proj = nn.Linear(dim, dim)
        self.proj_drop = nn.Dropout(proj_drop_ratio)

    def forward(self, x):
        # [batch_size, num_patches + 1, total_embed_dim]
        B, N, C = x.shape

        # qkv(): -> [batch_size, num_patches + 1, 3 * total_embed_dim]
        # reshape: -> [batch_size, num_patches + 1, 3, num_heads, embed_dim_per_head]
        # permute: -> [3, batch_size, num_heads, num_patches + 1, embed_dim_per_head]
        qkv = self.qkv(x).reshape(B, N, 3, self.num_heads, C // self.num_heads).permute(2,
0, 3, 1, 4)
```

```
        # [batch_size, num_heads, num_patches + 1, embed_dim_per_head]
        q, k, v = qkv[0], qkv[1], qkv[2]   # make torchscript happy (cannot use tensor as tuple)

        # transpose: -> [batch_size, num_heads, embed_dim_per_head, num_patches + 1]
        # @: multiply -> [batch_size, num_heads, num_patches + 1, num_patches + 1]
        attn = (q @ k.transpose(-2, -1)) * self.scale
        attn = attn.softmax(dim=-1)
        attn = self.attn_drop(attn)

        # @: multiply -> [batch_size, num_heads, num_patches + 1, embed_dim_per_head]
        # transpose: -> [batch_size, num_patches + 1, num_heads, embed_dim_per_head]
        # reshape: -> [batch_size, num_patches + 1, total_embed_dim]
        x = (attn @ v).transpose(1, 2).reshape(B, N, C)
        x = self.proj(x)
        x = self.proj_drop(x)
        return x


class Mlp(nn.Module):
    """
    MLP as used in Vision Transformer, MLP-Mixer and related networks
    """

    def __init__(self, in_features, hidden_features=None, out_features=None, act_layer=nn.GELU, drop=0.):
        super().__init__()
        out_features = out_features or in_features
        hidden_features = hidden_features or in_features
        self.fc1 = nn.Linear(in_features, hidden_features)
```

```
        self.act = act_layer()
        self.fc2 = nn.Linear(hidden_features, out_features)
        self.drop = nn.Dropout(drop)

    def forward(self, x):
        x = self.fc1(x)
        x = self.act(x)
        x = self.drop(x)
        x = self.fc2(x)
        x = self.drop(x)
        return x


class Block(nn.Module):
    def __init__(self,
                 dim,
                 num_heads,
                 mlp_ratio=4.,
                 qkv_bias=False,
                 qk_scale=None,
                 drop_ratio=0.,
                 attn_drop_ratio=0.,
                 drop_path_ratio=0.,
                 act_layer=nn.GELU,
                 norm_layer=nn.LayerNorm):
        super(Block, self).__init__()
        self.norm1 = norm_layer(dim)
        self.attn = Attention(dim, num_heads=num_heads, qkv_bias=qkv_bias,
qk_scale=qk_scale,
                              attn_drop_ratio=attn_drop_ratio,
```

```
proj_drop_ratio=drop_ratio)
        # NOTE: drop path for stochastic depth, we shall see if this is better than dropout here
        self.drop_path = DropPath(drop_path_ratio) if drop_path_ratio > 0. else nn.Identity()
        self.norm2 = norm_layer(dim)
        mlp_hidden_dim = int(dim * mlp_ratio)
        self.mlp = Mlp(in_features=dim, hidden_features=mlp_hidden_dim, act_layer=act_layer, drop=drop_ratio)

    def forward(self, x):
        x = x + self.drop_path(self.attn(self.norm1(x)))
        x = x + self.drop_path(self.mlp(self.norm2(x)))
        return x


class VisionTransformer(nn.Module):
    def __init__(self, img_size=224, patch_size=16, in_c=3, num_classes=1000,
                 embed_dim=768, depth=12, num_heads=12, mlp_ratio=4.0, qkv_bias=True,
                 qk_scale=None, representation_size=None, distilled=False, drop_ratio=0.,
                 attn_drop_ratio=0., drop_path_ratio=0., embed_layer=PatchEmbed, norm_layer=None,
                 act_layer=None):
        """
        Args:
            img_size (int, tuple): input image size
            patch_size (int, tuple): patch size
            in_c (int): number of input channels
```

```
            num_classes (int): number of classes for classification head
            embed_dim (int): embedding dimension
            depth (int): depth of transformer
            num_heads (int): number of attention heads
            mlp_ratio (int): ratio of mlp hidden dim to embedding dim
            qkv_bias (bool): enable bias for qkv if True
            qk_scale (float): override default qk scale of head_dim ** -0.5 if set
            representation_size (Optional[int]): enable and set representation layer (pre-logits) to this value if set
            distilled (bool): model includes a distillation token and head as in DeiT models
            drop_ratio (float): dropout rate
            attn_drop_ratio (float): attention dropout rate
            drop_path_ratio (float): stochastic depth rate
            embed_layer (nn.Module): patch embedding layer
            norm_layer: (nn.Module): normalization layer
        """
        super(VisionTransformer, self).__init__()
        self.num_classes = num_classes
        self.num_features = self.embed_dim = embed_dim  # num_features for consistency with other models
        self.num_tokens = 2 if distilled else 1
        norm_layer = norm_layer or partial(nn.LayerNorm, eps=1e-6)
        act_layer = act_layer or nn.GELU

        self.patch_embed = embed_layer(img_size=img_size, patch_size=patch_size, in_c=in_c, embed_dim=embed_dim)
        num_patches = self.patch_embed.num_patches

        self.cls_token = nn.Parameter(torch.zeros(1, 1, embed_dim))
        self.dist_token = nn.Parameter(torch.zeros(1, 1, embed_dim)) if distilled else None
```

```
        self.pos_embed = nn.Parameter(torch.zeros(1, num_patches + self.num_tokens, embed_dim))
        self.pos_drop = nn.Dropout(p=drop_ratio)

        dpr = [x.item() for x in torch.linspace(0, drop_path_ratio, depth)]  # stochastic depth decay rule
        self.blocks = nn.Sequential(*[
            Block(dim=embed_dim, num_heads=num_heads, mlp_ratio=mlp_ratio, qkv_bias=qkv_bias, qk_scale=qk_scale,
                  drop_ratio=drop_ratio, attn_drop_ratio=attn_drop_ratio, drop_path_ratio=dpr[i],
                  norm_layer=norm_layer, act_layer=act_layer)
            for i in range(depth)
        ])
        self.norm = norm_layer(embed_dim)

        # Representation layer
        if representation_size and not distilled:
            self.has_logits = True
            self.num_features = representation_size
            self.pre_logits = nn.Sequential(OrderedDict([
                ("fc", nn.Linear(embed_dim, representation_size)),
                ("act", nn.Tanh())
            ]))
        else:
            self.has_logits = False
            self.pre_logits = nn.Identity()

        # Classifier head(s)
        self.head = nn.Linear(self.num_features, num_classes) if num_classes > 0 else
```

```
nn.Identity()
        self.head_dist = None
        if distilled:
            self.head_dist = nn.Linear(self.embed_dim, self.num_classes) if num_classes >
0 else nn.Identity()

        # Weight init
        nn.init.trunc_normal_(self.pos_embed, std=0.02)
        if self.dist_token is not None:
            nn.init.trunc_normal_(self.dist_token, std=0.02)

        nn.init.trunc_normal_(self.cls_token, std=0.02)
        self.apply(_init_vit_weights)

    def forward_features(self, x):
        # [B, C, H, W] -> [B, num_patches, embed_dim]
        x = self.patch_embed(x)  # [B, 196, 768]
        # [1, 1, 768] -> [B, 1, 768]
        cls_token = self.cls_token.expand(x.shape[0], -1, -1)
        if self.dist_token is None:
            x = torch.cat((cls_token, x), dim=1)  # [B, 197, 768]
        else:
            x = torch.cat((cls_token, self.dist_token.expand(x.shape[0], -1, -1), x), dim=1)

        x = self.pos_drop(x + self.pos_embed)
        x = self.blocks(x)
        x = self.norm(x)
        if self.dist_token is None:
            return self.pre_logits(x[:, 0])
        else:
```

```
            return x[:, 0], x[:, 1]

    def forward(self, x):
        x = self.forward_features(x)
        if self.head_dist is not None:
            x, x_dist = self.head(x[0]), self.head_dist(x[1])
            if self.training and not torch.jit.is_scripting():
                # during inference, return the average of both classifier predictions
                return x, x_dist
            else:
                return (x + x_dist) / 2
        else:
            x = self.head(x)
        return x


def _init_vit_weights(m):
    """
    ViT weight initialization
    :param m: module
    """
    if isinstance(m, nn.Linear):
        nn.init.trunc_normal_(m.weight, std=.01)
        if m.bias is not None:
            nn.init.zeros_(m.bias)
    elif isinstance(m, nn.Conv2d):
        nn.init.kaiming_normal_(m.weight, mode="fan_out")
        if m.bias is not None:
            nn.init.zeros_(m.bias)
    elif isinstance(m, nn.LayerNorm):
```

```
            nn.init.zeros_(m.bias)
            nn.init.ones_(m.weight)


def vit_base_patch16_224(num_classes: int = 1000):
    """
    ViT-Base model (ViT-B/16) from original paper (https://arxiv.org/abs/2010.11929).
    ImageNet-1k weights @ 224x224, source https://github.com/google-research/vision_transformer.
    weights ported from official Google JAX impl:
    链接: https://pan.baidu.com/s/1zqb08naP0RPqqfSXfkB2EA  密码: eu9f
    """
    model = VisionTransformer(img_size=224,
                              patch_size=16,
                              embed_dim=768,
                              depth=12,
                              num_heads=12,
                              representation_size=None,
                              num_classes=num_classes)
    return model


def vit_base_patch16_224_in21k(num_classes: int = 21843, has_logits: bool = True):
    """
    ViT-Base model (ViT-B/16) from original paper (https://arxiv.org/abs/2010.11929).
    ImageNet-21k weights @ 224x224, source https://github.com/google-research/vision_transformer.
    weights ported from official Google JAX impl:

https://github.com/rwightman/pytorch-image-models/releases/download/v0.1-vitjx/jx_vit_bas
```

```
e_patch16_224_in21k-e5005f0a.pth
    """
    model = VisionTransformer(img_size=224,
                              patch_size=16,
                              embed_dim=768,
                              depth=12,
                              num_heads=12,
                              representation_size=768 if has_logits else None,
                              num_classes=num_classes)
    return model


def vit_base_patch32_224(num_classes: int = 1000):
    """
    ViT-Base model (ViT-B/32) from original paper (https://arxiv.org/abs/2010.11929).
    ImageNet-1k weights @ 224x224, source https://github.com/google-research/vision_transformer.
    weights ported from official Google JAX impl:
    链接: https://pan.baidu.com/s/1hCv0U8pQomwAtHBYc4hmZg  密码: s5hl
    """
    model = VisionTransformer(img_size=224,
                              patch_size=32,
                              embed_dim=768,
                              depth=12,
                              num_heads=12,
                              representation_size=None,
                              num_classes=num_classes)
    return model
```

```
def vit_base_patch32_224_in21k(num_classes: int = 21843, has_logits: bool = True):
    """
    ViT-Base model (ViT-B/32) from original paper (https://arxiv.org/abs/2010.11929).
    ImageNet-21k weights @ 224x224, source https://github.com/google-research/vision_transformer.
    weights ported from official Google JAX impl:

https://github.com/rwightman/pytorch-image-models/releases/download/v0.1-vitjx/jx_vit_base_patch32_224_in21k-8db57226.pth
    """
    model = VisionTransformer(img_size=224,
                              patch_size=32,
                              embed_dim=768,
                              depth=12,
                              num_heads=12,
                              representation_size=768 if has_logits else None,
                              num_classes=num_classes)
    return model


def vit_large_patch16_224(num_classes: int = 1000):
    """
    ViT-Large model (ViT-L/16) from original paper (https://arxiv.org/abs/2010.11929).
    ImageNet-1k weights @ 224x224, source https://github.com/google-research/vision_transformer.
    weights ported from official Google JAX impl:
    链接: https://pan.baidu.com/s/1cxBgZJJ6qUWPSBNcE4TdRQ  密码: qqt8
    """
    model = VisionTransformer(img_size=224,
                              patch_size=16,
```

```
                                    embed_dim=1024,
                                    depth=24,
                                    num_heads=16,
                                    representation_size=None,
                                    num_classes=num_classes)
    return model


def vit_large_patch16_224_in21k(num_classes: int = 21843, has_logits: bool = True):
    """
    ViT-Large model (ViT-L/16) from original paper (https://arxiv.org/abs/2010.11929).
    ImageNet-21k weights @ 224x224, source https://github.com/google-research/vision_transformer.
    weights ported from official Google JAX impl:

https://github.com/rwightman/pytorch-image-models/releases/download/v0.1-vitjx/jx_vit_large_patch16_224_in21k-606da67d.pth
    """
    model = VisionTransformer(img_size=224,
                                    patch_size=16,
                                    embed_dim=1024,
                                    depth=24,
                                    num_heads=16,
                                    representation_size=1024 if has_logits else None,
                                    num_classes=num_classes)
    return model


def vit_large_patch32_224_in21k(num_classes: int = 21843, has_logits: bool = True):
    """
```

```
    ViT-Large model (ViT-L/32) from original paper (https://arxiv.org/abs/2010.11929).
    ImageNet-21k weights @ 224x224, source https://github.com/google-research/vision_transformer.
    weights ported from official Google JAX impl:

https://github.com/rwightman/pytorch-image-models/releases/download/v0.1-vitjx/jx_vit_large_patch32_224_in21k-9046d2e7.pth
    """
    model = VisionTransformer(img_size=224,
                              patch_size=32,
                              embed_dim=1024,
                              depth=24,
                              num_heads=16,
                              representation_size=1024 if has_logits else None,
                              num_classes=num_classes)
    return model


def vit_huge_patch14_224_in21k(num_classes: int = 21843, has_logits: bool = True):
    """
    ViT-Huge model (ViT-H/14) from original paper (https://arxiv.org/abs/2010.11929).
    ImageNet-21k weights @ 224x224, source https://github.com/google-research/vision_transformer.
    NOTE: converted weights not currently available, too large for github release hosting.
    """
    model = VisionTransformer(img_size=224,
                              patch_size=14,
                              embed_dim=1280,
                              depth=32,
                              num_heads=16,
```

```
                                    representation_size=1280 if has_logits else None,
                                    num_classes=num_classes)
    return model
```

（2）train.py

```
# -*- coding: UTF-8 -*-
import math
import argparse
import os
import torch
import torch.optim as optim
import torch.optim.lr_scheduler as lr_scheduler
from torch.utils.tensorboard import SummaryWriter
from torchvision import transforms

from my_dataset import MyDataSet
from vit_model import vit_base_patch16_224_in21k as create_model
from utils import read_split_data, train_one_epoch, evaluate

from Config import train_path

import xlwt

book = xlwt.Workbook(encoding='utf-8')  # 创建 Workbook，相当于创建 Excel
# 创建 sheet，Sheet1 为表的名字，cell_overwrite_ok 为是否覆盖单元格
sheet1 = book.add_sheet(u'Train_data', cell_overwrite_ok=True)
# 向表中添加数据
sheet1.write(0, 0, 'epoch')
sheet1.write(0, 1, 'Train_Loss')
sheet1.write(0, 2, 'Train_Acc')
```

```
sheet1.write(0, 3, 'Val_Loss')
sheet1.write(0, 4, 'Val_Acc')
sheet1.write(0, 5, 'lr')
sheet1.write(0, 6, 'Best val Acc')


def main(args):
    best_acc = 0
    device = torch.device(args.device if torch.cuda.is_available() else "cpu")
    print(device)
    if os.path.exists("./weights") is False:
        os.makedirs("./weights")

    tb_writer = SummaryWriter()

    train_images_path, train_images_label, val_images_path, val_images_label = read_split_data(args.data_path)

    data_transform = {
        "train": transforms.Compose([transforms.RandomResizedCrop(224),
                                     transforms.RandomHorizontalFlip(),
                                     transforms.ToTensor(),
                                     transforms.Normalize([0.5, 0.5, 0.5], [0.5, 0.5, 0.5])]),
        "test": transforms.Compose([transforms.Resize(256),
                                    transforms.CenterCrop(224),
                                    transforms.ToTensor(),
                                    transforms.Normalize([0.5, 0.5, 0.5], [0.5, 0.5, 0.5])])}
```

```
# 实例化训练数据集
train_dataset = MyDataSet(images_path=train_images_path,
                          images_class=train_images_label,
                          transform=data_transform["train"])

# 实例化验证数据集
val_dataset = MyDataSet(images_path=val_images_path,
                        images_class=val_images_label,
                        transform=data_transform["test"])

batch_size = args.batch_size
nw = min([os.cpu_count(), batch_size if batch_size > 1 else 0, 8])  # number of workers
print('Using {} dataloader workers every process'.format(nw))
train_loader = torch.utils.data.DataLoader(train_dataset,
                                           batch_size=batch_size,
                                           shuffle=True,
                                           pin_memory=True,
                                           num_workers=nw,
                                           collate_fn=train_dataset.collate_fn)

val_loader = torch.utils.data.DataLoader(val_dataset,
                                         batch_size=batch_size,
                                         shuffle=False,
                                         pin_memory=True,
                                         num_workers=nw,
                                         collate_fn=val_dataset.collate_fn)

images = torch.zeros(1, 3, 224, 224).to('cuda')  # 要求大小与输入图片的大小一致
model = create_model(num_classes=2, has_logits=False).to(device)
```

```
    images = torch.zeros(1, 3, 224, 224).to(device)  # 要求大小与输入图片的大小一致
    tb_writer.add_graph(model, images, verbose=False)

    if args.weights != "":
        assert os.path.exists(args.weights), "weights file: '{}' not exist.".format(args.weights)
        weights_dict = torch.load(args.weights, map_location=device)
        # 删除不需要的权重
        del_keys = ['head.weight', 'head.bias'] if model.has_logits \
            else ['pre_logits.fc.weight', 'pre_logits.fc.bias', 'head.weight', 'head.bias']
        for k in del_keys:
            del weights_dict[k]
        print(model.load_state_dict(weights_dict, strict=False))

    if args.freeze_layers:
        for name, para in model.named_parameters():
            # 除 head, pre_logits 外，其他权重全部冻结
            if "head" not in name and "pre_logits" not in name:
                para.requires_grad_(False)
            else:
                print("training {}".format(name))

    pg = [p for p in model.parameters() if p.requires_grad]

    optimizer = optim.SGD(pg, lr=args.lr, momentum=0.9, weight_decay=5E-5)

    lf = lambda x: ((1 + math.cos(x * math.pi / args.epochs)) / 2) * (1 - args.lrf) + args.lrf  # cosine
    scheduler = lr_scheduler.LambdaLR(optimizer, lr_lambda=lf)
```

```
for epoch in range(args.epochs):

    sheet1.write(epoch + 1, 0, epoch + 1)
    sheet1.write(epoch + 1, 5, str(optimizer.state_dict()['param_groups'][0]['lr']))

    # train
    train_loss, train_acc = train_one_epoch(model=model,
                                            optimizer=optimizer,
                                            data_loader=train_loader,
                                            device=device,
                                            epoch=epoch)

    scheduler.step()

    sheet1.write(epoch + 1, 1, str(train_loss))
    sheet1.write(epoch + 1, 2, str(train_acc))

    # validate
    val_loss, val_acc = evaluate(model=model,
                                 data_loader=val_loader,
                                 device=device,
                                 epoch=epoch)

    sheet1.write(epoch + 1, 3, str(val_loss))
    sheet1.write(epoch + 1, 4, str(val_acc))

    tags = ["train_loss", "train_acc", "val_loss", "val_acc", "learning_rate"]
    tb_writer.add_scalar(tags[0], train_loss, epoch)
    tb_writer.add_scalar(tags[1], train_acc, epoch)
    tb_writer.add_scalar(tags[2], val_loss, epoch)
```

```
            tb_writer.add_scalar(tags[3], val_acc, epoch)
            tb_writer.add_scalar(tags[4], optimizer.param_groups[0]["lr"], epoch)

            if val_acc > best_acc:
                best_acc = val_acc
                torch.save(model.state_dict(),
r"./weights/epoch{}_batch{}_model_1102.pth".format(args.epochs, batch_size))
                # torch.save(model.state_dict(), "./weights/model_1101.pth")

        sheet1.write(1, 6, str(best_acc))

book.save(r'/work/mathorcup-codeA-1.0/result/epoch{}_batch{}_Train_data_1102.xlsx'.form
at(args.epochs, batch_size))
        print("The Best Acc = : {:.4f}".format(best_acc))


if __name__ == '__main__':
        parser = argparse.ArgumentParser()
        parser.add_argument('--num_classes', type=int, default=2)
        parser.add_argument('--epochs', type=int, default=10)
        parser.add_argument('--batch-size', type=int, default=5)
        parser.add_argument('--lr', type=float, default=0.001)
        parser.add_argument('--lrf', type=float, default=0.01)

        # 数据集所在根目录
        parser.add_argument('--data-path', type=str,
                                    default=train_path)
        parser.add_argument('--model-name', default='', help='create model name')

        # 预训练权重路径，如果不想载入就设置为空字符
```

```
    parser.add_argument('--weights', type=str,

default=r"/work/mathorcup-codeA-1.0/pre_model/vit_base_patch16_224_in21k.pth",
                                help='initial weights path')

    # 是否冻结权重
    parser.add_argument('--freeze-layers', type=bool, default=False)
    parser.add_argument('--device', default='cuda:1', help='device id (i.e. 0 or 0,1 or cpu)')

    opt = parser.parse_args()

    main(opt)
```

（3）utils.py

```
# -*- coding: UTF-8 -*-
import os
import sys
import json
import pickle
import random

import torch
from tqdm import tqdm

import matplotlib.pyplot as plt


def read_split_data(root: str, val_rate: float = 0.2):
    random.seed(0)  # 保证随机结果可复现
    assert os.path.exists(root), "dataset root: {} does not exist.".format(root)
```

```
# 遍历文件夹，一个文件夹对应一个类别
flower_class = [cla for cla in os.listdir(root) if os.path.isdir(os.path.join(root, cla))]
# 排序，保证顺序一致
flower_class.sort()
# 生成类别名称以及对应的数字索引
class_indices = dict((k, v) for v, k in enumerate(flower_class))
json_str = json.dumps(dict((val, key) for key, val in class_indices.items()), indent=4)
with open('class_indices.json', 'w') as json_file:
    json_file.write(json_str)

train_images_path = []  # 存储训练集的所有图片路径
train_images_label = []  # 存储训练集图片对应索引信息
val_images_path = []  # 存储验证集的所有图片路径
val_images_label = []  # 存储验证集图片对应索引信息
every_class_num = []  # 存储每个类别的样本总数
supported = [".jpg", ".JPG", ".png", ".PNG"]  # 支持的文件后缀类型
# 遍历每个文件夹下的文件
for cla in flower_class:
    cla_path = os.path.join(root, cla)
    # 遍历获取 supported 支持的所有文件路径
    images = [os.path.join(root, cla, i) for i in os.listdir(cla_path)
              if os.path.splitext(i)[-1] in supported]
    # 获取该类别对应的索引
    image_class = class_indices[cla]
    # 记录该类别的样本数量
    every_class_num.append(len(images))
    # 按比例随机采样验证样本
    val_path = random.sample(images, k=int(len(images) * val_rate))
```

```
        for img_path in images:
            if img_path in val_path:  # 如果该路径在采样的验证集样本中则存入验证集
                val_images_path.append(img_path)
                val_images_label.append(image_class)
            else:  # 否则存入训练集
                train_images_path.append(img_path)
                train_images_label.append(image_class)

    print("{} images were found in the dataset.".format(sum(every_class_num)))
    print("{} images for training.".format(len(train_images_path)))
    print("{} images for validation.".format(len(val_images_path)))

    plot_image = False
    if plot_image:
        # 绘制每种类别个数柱状图
        plt.bar(range(len(flower_class)), every_class_num, align='center')
        # 将横坐标 0,1,2,3,4 替换为相应的类别名称
        plt.xticks(range(len(flower_class)), flower_class)
        # 在柱状图上添加数值标签
        for i, v in enumerate(every_class_num):
            plt.text(x=i, y=v + 5, s=str(v), ha='center')
        # 设置 x 坐标
        plt.xlabel('image class')
        # 设置 y 坐标
        plt.ylabel('number of images')
        # 设置柱状图的标题
        plt.title('flower class distribution')
        plt.show()
```

```
    return train_images_path, train_images_label, val_images_path, val_images_label


def plot_data_loader_image(data_loader):
    batch_size = data_loader.batch_size
    plot_num = min(batch_size, 4)

    json_path = './class_indices.json'
    assert os.path.exists(json_path), json_path + " does not exist."
    json_file = open(json_path, 'r')
    class_indices = json.load(json_file)

    for data in data_loader:
        images, labels = data
        for i in range(plot_num):
            # [C, H, W] -> [H, W, C]
            img = images[i].numpy().transpose(1, 2, 0)
            #  反 Normalize 操作
            img = (img * [0.229, 0.224, 0.225] + [0.485, 0.456, 0.406]) * 255
            label = labels[i].item()
            plt.subplot(1, plot_num, i + 1)
            plt.xlabel(class_indices[str(label)])
            plt.xticks([])   #  去掉 x 轴的刻度
            plt.yticks([])   #  去掉 y 轴的刻度
            plt.imshow(img.astype('uint8'))
        plt.show()


def write_pickle(list_info: list, file_name: str):
    with open(file_name, 'wb') as f:
```

```
        pickle.dump(list_info, f)


def read_pickle(file_name: str) -> list:
    with open(file_name, 'rb') as f:
        info_list = pickle.load(f)
        return info_list


def train_one_epoch(model, optimizer, data_loader, device, epoch):
    model.train()
    loss_function = torch.nn.CrossEntropyLoss()
    accu_loss = torch.zeros(1).to(device)  # 累计损失
    accu_num = torch.zeros(1).to(device)  # 累计预测正确的样本数
    optimizer.zero_grad()

    sample_num = 0
    data_loader = tqdm(data_loader, file=sys.stdout)
    for step, data in enumerate(data_loader):
        images, labels = data
        sample_num += images.shape[0]

        pred = model(images.to(device))
        pred_classes = torch.max(pred, dim=1)[1]
        accu_num += torch.eq(pred_classes, labels.to(device)).sum()

        loss = loss_function(pred, labels.to(device))
        loss.backward()
        accu_loss += loss.detach()
```

```
            data_loader.desc = "[train epoch {}] loss: {:.3f}, acc: {:.3f}".format(epoch,
accu_loss.item() / (step + 1),
accu_num.item() / sample_num)

            if not torch.isfinite(loss):
                print('WARNING: non-finite loss, ending training ', loss)
                sys.exit(1)

            optimizer.step()
            optimizer.zero_grad()

        return accu_loss.item() / (step + 1), accu_num.item() / sample_num


@torch.no_grad()
def evaluate(model, data_loader, device, epoch):
        loss_function = torch.nn.CrossEntropyLoss()

        model.eval()

        accu_num = torch.zeros(1).to(device)   # 累计预测正确的样本数
        accu_loss = torch.zeros(1).to(device)   # 累计损失

        sample_num = 0
        data_loader = tqdm(data_loader, file=sys.stdout)
        for step, data in enumerate(data_loader):
            images, labels = data
            sample_num += images.shape[0]
```

```
            pred = model(images.to(device))
            pred_classes = torch.max(pred, dim=1)[1]
            accu_num += torch.eq(pred_classes, labels.to(device)).sum()

            loss = loss_function(pred, labels.to(device))
            accu_loss += loss

            data_loader.desc = "[valid epoch {}] loss: {:.3f}, acc: {:.3f}".format(epoch,

accu_loss.item() / (step + 1),

accu_num.item() / sample_num)

        return accu_loss.item() / (step + 1), accu_num.item() / sample_num
```

（4）dataset.py

```
# -*- coding: UTF-8 -*-
from PIL import Image

import torch
from torch.utils.data import Dataset


class MyDataSet(Dataset):
    """自定义数据集"""

    def __init__(self, images_path: list, images_class: list, transform=None):
        self.images_path = images_path
        self.images_class = images_class
```

```
        self.transform = transform

    def __len__(self):
        return len(self.images_path)

    def __getitem__(self, item):
        img = Image.open(self.images_path[item])
        # RGB 为彩色图片，L 为灰度图片
        if img.mode != 'RGB':
            raise ValueError("image: {} isn't RGB mode.".format(self.images_path[item]))
        label = self.images_class[item]

        if self.transform is not None:
            img = self.transform(img)

        return img, label

    @staticmethod
    def collate_fn(batch):
        images, labels = tuple(zip(*batch))

        images = torch.stack(images, dim=0)
        labels = torch.as_tensor(labels)
        return images, labels
```

（5）test_data_predict.py

```
# -*- coding:utf-8 -*-
import json
import random
import pickle
```

```
from Config import model_path
import os

import matplotlib.pyplot as plt
import torch
from PIL import Image
from torchvision import transforms

from vit_model import vit_base_patch16_224_in21k as create_model

from alive_progress import alive_bar


# 数据预处理
def get_label(file_name):
    return 1 if "normal" in file_name else 0


def get_test_dataset(test_path):
    test_dataset = [pic for pic in os.listdir(test_path)]
    return test_dataset


def pre_process():
    device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

    data_transform = transforms.Compose(
        [transforms.Resize(256),
         transforms.CenterCrop(224),
         transforms.ToTensor(),
```

```
            transforms.Normalize([0.5, 0.5, 0.5], [0.5, 0.5, 0.5])])
    # read class_indict
    json_path = './class_indices.json'
    assert os.path.exists(json_path), "file: '{}' dose not exist.".format(json_path)

    with open(json_path, "r") as f:
        class_indict = json.load(f)

    # create model
    model = create_model(num_classes=2, has_logits=False).to(device)
    # load model weights
    model_weight_path = model_path
    model.load_state_dict(torch.load(model_weight_path, map_location=device))
    model.eval()

    return model, class_indict, device, data_transform


def model_predict(test_pic, pic_path, model, class_indict, device, data_transform):
    # load image
    img_path = pic_path
    assert os.path.exists(img_path), "file: '{}' dose not exist.".format(img_path)
    img = Image.open(img_path)
    # plt.imshow(img)
    # [N, C, H, W]
    img = data_transform(img)
    # expand batch dimension
    img = torch.unsqueeze(img, dim=0)

    with torch.no_grad():
```

```
            # predict class
            output = torch.squeeze(model(img.to(device))).cpu()
            predict = torch.softmax(output, dim=0)
            predict_cla = torch.argmax(predict).numpy()

        # for i in range(len(predict)):
        #         print("class: {:10}      prob: {:.3}".format(class_indict[str(i)], predict[i].numpy()))

        normal_probability = float(predict[0].numpy())
        potholes_probability = float(predict[1].numpy())

        if normal_probability >= potholes_probability:
            predict_label = 1
        else:
            predict_label = 0
        if abs(normal_probability - potholes_probability) <= 0.3:
            for i in range(len(predict)):
                print("class:      {:10}                prob:      {:.3}".format(class_indict[str(i)],
predict[i].numpy()))
            print(pic_path)
            print("预测结果：", predict_label)
            print("--------------------------")

        # 绘制图像
        print_res = "name:{}        label:{}        class: {}        prob: {:.3}".format(test_pic,
predict_label,

class_indict[str(predict_cla)],

predict[predict_cla].numpy())
```

```
    # plt.title(print_res)
    # plt.savefig(r"/work/mathorcup-codeA-1.0/result/test_data/result_" + test_pic)

    return predict_label, normal_probability, potholes_probability


def main():
    test_path = '/work/mathorcup-codeA-1.0/test_data/testdata_V2'
    test_dataset = get_test_dataset(test_path)
    predict_dict = {}

    error_images = []
    error_predicts = []

    # 调用模型
    model, class_indict, device, data_transform = pre_process()

    with alive_bar(len(test_dataset)) as bar:
        for test_pic in test_dataset:
            try:
                pic_path = test_path + '/' + test_pic
                predict_label, normal_probability, potholes_probability =
model_predict(test_pic, pic_path, model,

class_indict,

device, data_transform)
                predict_dict[test_pic] = predict_label
                # print("预测结果: ", predict_label)
```

```
                except:
                    print(test_pic, "图片处理失败")
                    error_images.append(test_pic)

                # 显示进度条
                bar()
        print("处理失败图片为:\n", error_images)
        # print("写入 csv 的字典:\n", predict_dict)

        # 生成写入问题 3 内.csv 文件的字典文件
        with open('/work/mathorcup-codeA-1.0/result/save_csv_data/save_csv_data.pkl', 'wb') as
f:
            pickle.dump(predict_dict, f)


if __name__ == '__main__':
    main()
```